

Práctica 14. Laberinto de 72 estados

Garay Pérez Matthew Ilveneff

202210036@tesco.edu.mx

Tecnológico de Estudios Superiores
de Coacalco

Ingeniería en Sistemas
Computacionales

Programación Lógica Y Funcional

Ing. Vivanco Benavides Luis Enrique

Entrega 06 de diciembre de 2025

3721

Introducción

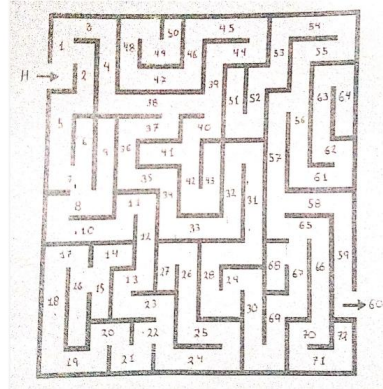
El desafío de encontrar un camino viable entre un punto inicial y un punto final dentro de un espacio delimitado, como un laberinto, es un problema clásico de la Teoría de Grafos y la Inteligencia Artificial (Russell & Norvig, 2021). Este tipo de problemas se resuelven eficientemente modelando el espacio como un grafo, donde las intersecciones o áreas numeradas actúan como nodos y los caminos que las conectan actúan como aristas. Este trabajo presenta una implementación en el lenguaje de Programación Lógica Prolog para resolver un laberinto bidimensional complejo, utilizando específicamente el algoritmo de Búsqueda en Profundidad (DFS).

La Búsqueda en Profundidad se distingue por su estrategia de exploración exhaustiva, donde prioriza la profundidad sobre la amplitud, explorando completamente una rama del grafo antes de retroceder (backtracking) para examinar la siguiente (Covington et al., 1997). El código fuente desarrollado en Prolog codifica el laberinto como un conjunto de listas de adyacencia, lo que permite al motor de inferencia del lenguaje buscar automáticamente las relaciones transitivas que definen un camino.

El objetivo principal de este estudio es validar la capacidad de Prolog para manejar problemas de búsqueda de estados (como la solución de laberintos) de forma declarativa y eficiente. La implementación se centra en los predicados `busqueda/4` y `camino1/5`, que gestionan la exploración y la prevención de ciclos mediante el seguimiento de nodos visitados. Al final, el predicado `resolver_labirinto/3` proporcionará el camino secuencial de nodos desde el inicio ('H') hasta la salida ('n60'), demostrando así el poder de los algoritmos de búsqueda basados en reglas.

Desarrollo

La primera etapa crucial consistió en la Modelación del Grafo, donde la estructura visual del laberinto, con sus nodos numerados y conexiones, fue traducida a una lista de adyacencia en Prolog. Esta lista, denominada \$G\$ en el código, representa cada nodo (desde la entrada 'h' hasta la salida 'n60') y todos sus vecinos directos, sentando la base para la búsqueda lógica.



```
resolver4(Ni, Nf, P) :- busqueda( Ni, Nf,
[[n1, n2, n3], [n2, n1, n5], [n3, n1, n4], [n4, n3, n38],
[n5, n2, n7], [n6, n7], [n7, n5, n6, n8], [n8, n7, n9, n10],
[n9, n8], [n10, n8, n11], [n11, n10, n12],
[n12, n11, n13], [n13, n12, n23], [n14, n15],
[n15, n16, n14], [n16, n15, n17], [n17, n16, n18],
[n18, n17, n19], [n19, n18, n20], [n20, n19, n21], [n21, n20, n22],
[n22, n21, n23, n24], [n23, n13, n22], [n24, n22, n25], [n25, n24, n26],
[n26, n25, n27], [n27, n26], [n28, n29, n31], [n29, n28, n30], [n30, n29, n69],
[n31, n28, n32], [n32, n31, n33], [n33, n32, n34], [n34, n33, n35],
[n35, n34, n36], [n36, n35, n37], [n37, n36, n38], [n38, n4, n37, n39, n40],
[n39, n38, n40, n44], [n40, n38, n39, n41, n42], [n41, n40, n42],
[n42, n40, n41, n43], [n43, n42], [n44, n39, n45], [n45, n46, n44],
[n46, n45, n47], [n47, n46, n48], [n48, n47, n49], [n49, n48, n50],
[n50, n49], [n51, n52], [n52, n51, n53], [n53, n52, n54],
[n54, n53, n55], [n55, n54, n56], [n56, n55, n57, n61], [n57, n56, n58, n65],
[n58, n57, n59, n65], [n59, n58, n60], [n60, n59], [n61, n56, n62], [n62, n61, n63],
[n63, n62, n64], [n64, n63], [n65, n57, n66, n67, n68], [n66, n65, n70], [n67, n65, n68, n69],
[n68, n65, n67], [n69, n30, n67], [n70, n66, n71], [n71, n70, n72], [n72, n72], P].
```

El código opera en tres fases. En la inicialización y búsqueda (busqueda/4), el código comienza el proceso desde el nodo de inicio (\$A\$), llamando al predicado recursivo camino1. Este es el núcleo del recorrido profundo (camino1/5), que realiza la exploración: desde el nodo actual (\$X\$), busca un sucesor disponible (\$Y\$) utilizando sucesor_disp. Este predicado es clave porque, además de verificar la adyacencia, implementa la restricción anti-ciclo (not(miembro(Y, P1))), asegurando que el algoritmo solo avance hacia nodos que aún no han sido visitados en el camino actual. Al encontrar un sucesor, el algoritmo profundiza (camino1(Y, Z, [Y | P1], G, P)) agregando el nuevo nodo al inicio de la lista del camino parcial. La recursión

Dependiendo que camino tomes te puede dar así como el camino largo y no tan largo en este caso creo que este es el camino mas viable para llegar de 1 a 6° donde termina el laberinto.

```
?- resolver(n1,n60,X).
X = [n1, n3, n4, n38, n37, n36, n35, n34, n33|...]
Unknown action: M (h for help)
Action? .

?- resolver(n33,n60,X).
X = [n33, n32, n31, n28, n29, n30, n69, n67, n65|...] ;
X = [n33, n32, n31, n28, n29, n30, n69, n67, n68|...] ;
false.

?- resolver(n65,n60,X).
X = [n65, n57, n58, n59, n60] ;
false.
```

Conclusiones

La conclusión de esta práctica reside en cómo el programa Prolog simula una estrategia humana de exploración dentro del laberinto. El código implementa la Búsqueda en Profundidad (DFS), que imita el proceso intuitivo de una persona que decide "ir por un camino y seguirlo hasta el final". Esta aproximación garantiza que la solución sea encontrada si existe, pero no está optimizada para la eficiencia. Al igual que un humano que se pierde en una sección sin darse cuenta de que la salida estaba cerca, el DFS prioriza la profundidad de la exploración; por ello, la consulta puede devolver el camino más largo que satisfaga la condición de salida, ya que es la primera ruta que el sistema encuentra antes de tener que retroceder y probar alternativas. El programa se comporta, así como un explorador lógico que utiliza una memoria perfecta (la función anti-ciclo) y un mecanismo de retroceso automático (el backtracking de Prolog) para encontrar la ruta de forma exhaustiva, traduciendo una pregunta espacial compleja en una secuencia lógica de movimientos, demostrando la capacidad de la programación lógica para modelar el razonamiento estratégico.

Referencias Bibliográficas

Covington, M. A., Nute, D., & Vellino, A. (1997). Prolog programming in depth. Prentice Hall.

Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson Education.